

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory work No. 9

Watching System Calls with strace

Coșneanu Corina, st. gr. FAF-241

Checked by:
Reițman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Contents

1 LABORATORY OBJECTIVES AND THEORY **1**
1.1 Objective 1
1.2 Theoretical Background 1

2 PRACTICAL WORK AND OBSERVATIONS **2**
2.1 Part 1: See the calls of a real command 2
2.1.1 Observe Answers (Part 1) 3
2.2 Part 2: A program that calls the kernel directly 4
2.2.1 Observe Answers (Part 2) 5
2.3 Part 3: Count one kind of call 5

3 CONCLUSIONS **6**

1 LABORATORY OBJECTIVES AND THEORY

1.1 Objective

The main objective of this laboratory work is to monitor and analyze the system calls executed by both standard Linux utilities and custom C programs using the `strace` diagnostic tool. This provides a practical understanding of the kernel-user space boundary and file descriptor allocation.

1.2 Theoretical Background

A user-space program runs in an unprivileged mode and cannot access hardware or kernel data directly. When it needs a service from the operating system (such as reading a file or printing to the screen), it must cross the boundary into the kernel via a **system call**. The kernel performs the requested operation in privileged mode and returns the result to the program.

- **strace:** A Linux utility that intercepts and records the system calls made by a process and the signals received by it. The options used in this work are `-f` (also follow child processes), `-e trace=...` (show only the listed system calls) and `-c` (print a summary table with the number of calls instead of the full trace).
- **File descriptors:** Small non-negative integers used by the kernel to identify the files a process has open. Every process starts with three descriptors already open, inherited from its parent (usually the shell, which connects them to the terminal): `0` (standard input), `1` (standard output) and `2` (standard error). When a process opens a file, the kernel returns the lowest unused descriptor number.
- **open, read, write, close:** The basic file system calls: `open` returns a descriptor for a file, `read` and `write` transfer bytes through that descriptor, and `close` releases it. In C, these are thin wrapper functions from the C library (libc). On modern Linux, the `open()` wrapper uses the `openat` system call, which is why `strace` shows `openat` instead of `open`.

2 PRACTICAL WORK AND OBSERVATIONS

2.1 Part 1: See the calls of a real command

By running `strace -f -e trace=openat,read,write,close ls -l`, we can observe the hidden interactions between the utility and the kernel. Figure **??** shows the beginning of the trace: before listing anything, the process opens and reads several shared libraries (the loader cache, `libselinux`, `libc` and others) and some system files from `/proc`. Figure 2.1 shows the end of the trace: one lookup for an `ls`-specific translation file fails with `ENOENT` (a harmless error), then the directory is opened with `openat`, closed, and the result is written with a single `write` call to descriptor 1, after which the process exits with status 0.

```
vboxuser@lab1:~/os-2026-CosneanuCorina/lab9$ ls
note.txt  report.md  syscalls.c
vboxuser@lab1:~/os-2026-CosneanuCorina/lab9$ strace -f -e trace=openat,read,writ
e,close ls -l 2>&1 | tail -n 15
openat(AT_FDCWD, "/usr/share/zoneinfo/America/Indiana", O_RDONLY|O_NONBLOCK|O_CL
OEXEC|O_DIRECTORY) = 4
close(4)                                = 0
openat(AT_FDCWD, "/usr/share/zoneinfo/America/Kentucky", O_RDONLY|O_NONBLOCK|O_C
LOEXEC|O_DIRECTORY) = 4
close(4)                                = 0
openat(AT_FDCWD, "/usr/share/zoneinfo/Etc/UTC", O_RDONLY|O_CLOEXEC) = 4
read(4, "TZif2\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"..., 114) =
 114
read(4, "", 32)                         = 0
close(4)                                = 0
close(3)                                = 0
write(1, "total 12\n-rw-rw-r-- 1 vboxuser v"..., 183total 12
-rw-rw-r-- 1 vboxuser vboxuser  14 Nov  9 13:19 note.txt
-rw-rw-r-- 1 vboxuser vboxuser 966 Nov  9 13:18 report.md
-rw-rw-r-- 1 vboxuser vboxuser 595 Nov  9 13:18 syscalls.c
) = 183
+++ exited with 0 +++
```

Figure 2.1: `strace` output for `ls -l`: opening the directory and writing the result

To estimate how many system calls a simple `ls` makes, we used `strace -c ls`, which counts all system calls, not only the four filtered above. Figure 2.2 shows that `ls` made **137 system calls** in total, of which 5 returned an error. The most frequent calls are `mmap` (27), `read` (14), `openat` (11) and `close` (10), which come mostly from loading the libraries. The whole output of `ls` is produced by a single `write`, while `execve`, a single call, takes over half of the total time.

```
vboxuser@lab1:~/xv6-riscv/os-lab9$ strace -c ls
note.txt  report.md  syscalls  syscalls.c
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 55.27    0.003531        3531         1           execve
 11.66    0.000745          67        11         1 openat
 10.33    0.000660         110         6           statx
  5.07    0.000324          32        10           close
  2.79    0.000178          35         5           brk
  2.71    0.000173           6        27           mmap
  2.44    0.000156          11        14           read
  2.10    0.000134          67         2           munmap
  1.78    0.000114          57         2           prlimit64
  1.75    0.000112          56         2           getdents64
  1.69    0.000108          13         8           mprotect
  0.34    0.000022           2         9           fstat
  0.34    0.000022           2         9           rt_sigaction
  0.30    0.000019           9         2           lseek
  0.22    0.000014           2         6           ioctl
  0.19    0.000012          12         1           write
  0.19    0.000012           6         2         2 access
  0.14    0.000009           3         3           sigaltstack
  0.13    0.000008           2         3           fcntl
  0.09    0.000006           6         1           poll
  0.06    0.000004           2         2           pread64
  0.06    0.000004           4         1           prctl
  0.06    0.000004           4         1           sched_getaffinity
  0.05    0.000003           3         1           gettid
  0.05    0.000003           3         1           set_robust_list
  0.05    0.000003           3         1           getrandom
  0.05    0.000003           3         1           rseq
  0.03    0.000002           2         1           arch_prctl
  0.03    0.000002           2         1           set_tid_address
  0.03    0.000002           2         1           newfstatat
  0.00    0.000000           0         2         2 statfs
------ ----------- ----------- --------- --------- ----------------
100.00    0.006389          46       137         5 total
```

Figure 2.2: System call summary of `ls` produced by `strace -c ls`: 137 calls in total

2.1.1 Observe Answers (Part 1)

1. **System Call for Printing:** The `ls` command uses the `write` system call to send its output to the screen (see the last `write` in Figure 2.1).

2. **Target File Descriptor:** The output is written to file descriptor `1`, which represents the standard output (`stdout`), as shown by the first argument of `write(1, ...)`.

3. **Number of Calls:** A simple `ls` makes about 137 system calls (Figure 2.2).

2.2 Part 2: A program that calls the kernel directly

A custom C program (`syscalls.c`) was written to do file input/output directly through the system call interface, using `open`, `write`, `read` and `close` instead of the buffered functions of the standard library (such as `fopen` or `fprintf`). These functions are thin wrappers around the kernel's system calls. The program creates the file `note.txt`, writes one line into it, and reads the line back:

```c
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <stdio.h>

int main(void) {
    const char *msg = "hello, kernel\n";

    int fd = open("note.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd < 0) { perror("open"); return 1; }
    printf("open returned descriptor %d\n", fd);

    ssize_t n = write(fd, msg, strlen(msg));
    printf("write sent %zd bytes\n", n);
    close(fd);

    fd = open("note.txt", O_RDONLY);
    char buf[64];
    n = read(fd, buf, sizeof(buf) - 1);
    buf[n] = '\0';
    printf("read got %zd bytes: %s", n, buf);
    close(fd);

    return 0;
}
```

The program was compiled and run first normally and then under `strace`:

```
$ gcc -Wall -Wextra -o syscalls syscalls.c
$ ./syscalls
$ strace -e trace=openat,write,read,close ./syscalls
```

The result is shown in Figure 2.3. The output of the program matches the expected one (descriptor 3, 14 bytes written, 14 bytes read), and the three C calls can be found in the trace:

```
open("note.txt", O_CREAT|O_WRONLY|O_TRUNC, 0644)
  -> openat(AT_FDCWD, "note.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(fd, msg, 14)
  -> write(3, "hello, kernel\n", 14) = 14
read(fd, buf, 63)
  -> read(3, "hello, kernel\n", 63) = 14
```

In the `read` call, 63 is the size of the buffer minus one byte. Each `printf` produces its own `write(1, ...)`, with 27, 20 and 33 bytes, one for each line printed on the terminal. The first lines of the trace (`/etc/ld.so.cache`, `libc.so.6`) belong to the dynamic loader, which starts every dynamically linked program.

```
vboxuser@lab1:~/os-lab9$ ./syscalls
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
vboxuser@lab1:~/os-lab9$ strace -e trace=openat,write,read,close ./syscalls
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0 \250\2\0\0\0\0\0"..., 832) = 832
close(3)                                = 0
openat(AT_FDCWD, "note.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(1, "open returned descriptor 3\n", 27open returned descriptor 3
) = 27
write(3, "hello, kernel\n", 14)         = 14
write(1, "write sent 14 bytes\n", 20write sent 14 bytes
)   = 20
close(3)                                = 0
openat(AT_FDCWD, "note.txt", O_RDONLY)  = 3
read(3, "hello, kernel\n", 63)          = 14
write(1, "read got 14 bytes: hello, kernel"..., 33read got 14 bytes: hello, kernel
) = 33
close(3)                                = 0
+++ exited with 0 +++
```

Figure 2.3: Execution and system call trace of the custom `syscalls` program

2.2.1 Observe Answers (Part 2)

1. **First Available Descriptor:** The descriptor returned by the first `openat` call for `note.txt` is `3`, rather than `0`, `1` or `2`. (The earlier `openat` calls of the loader also return 3, but those files are closed immediately.)

2. **Pre-allocated Descriptors:** Descriptors `0`, `1` and `2` are already taken because every process starts with them open: they are inherited from the parent process (here the shell), which connected them to the terminal as standard input (`stdin`), standard output (`stdout`) and standard error (`stderr`), respectively. The kernel always returns the lowest unused descriptor, so the first free one is `3`. After `close(3)`, the number is free again, which is why the second `openat` also returns `3`.

2.3 Part 3: Count one kind of call

Counting only one kind of call makes the behavior of the program easier to see. In the trace of `syscalls` (Figure 2.3) there are **four** `write` calls: three to descriptor 1, one for each `printf` line, and one to descriptor 3, which puts the text into `note.txt`. There are also four `openat` calls (two for the loader and two for `note.txt`). For comparison, `ls` needs only one `write` for its whole output (Figure 2.2), but eleven `openat` calls.

The three separate `write(1, ...)` calls appear because the standard output is a terminal, so the C library flushes the buffer at every newline (line buffering). If the output is redirected to a file or a pipe, the library buffers it and sends all lines in one single `write`, so the same program makes only two `write` calls in that case. The count of one kind of call can also be obtained directly with a filter, for example `strace -c -e trace=write ./syscalls`.

3 CONCLUSIONS

In this laboratory session, I used the `strace` tool to observe the kernel-level operations of programs running on a Linux system. Analyzing a standard command like `ls` showed that its terminal output relies on the `write` system call targeting file descriptor 1, and that even this simple command makes 137 system calls, most of them to load the shared libraries.

Furthermore, writing and tracing a custom C program (`syscalls.c`) showed how the functions `open`, `write`, `read` and `close` become real system calls (`open` appears as `openat`), and how file descriptors are allocated: indices 0, 1 and 2 are inherited from the parent process, so the kernel gives index 3, the lowest unused one, to the first newly opened file. Counting the calls also showed that the number of `write` calls depends on buffering: the terminal output of `printf` is flushed line by line, which produces separate `write` calls.